Code Jam 2010 - Round 1A

Analysis: Make it Smooth

The Basic Solution

Just about any solution to this problem is going to ultimately rely on building up a smoothed array by first solving smaller sub-problems. The challenge is two-fold: (1) What are the sub-problems you want to solve? And (2) How can you efficiently process them all?

It is natural to start off by making the first `N-1` pixels smooth, and then figuring out afterwards what to do with the last pixel. The catch is what we do with the last pixel `q` depends on `p`, the final pixel we set before-hand. If `p` and `q` differ by at most `M`, then we are already done! Otherwise, we have two choices:

- Delete `q` with cost `D`, leaving the final pixel of our smoothed picture as `p`.
- Move `q` to some value `q'` with cost `|q - q'|`. If `|q' - p| > M`, we will then have to add some pixels before-hand to make the transition smooth. In fact, we will need to add exactly `(|q' - p| - 1)/ M` of these pixels.

Fortunately, both of these cases are easy to analyze, as long as we are willing to loop through every possible value of `q'`.

Perhaps the trickiest part of this setup is understanding insertions. After all, when deciding what steps to take to smooth out the transition from one pixel in the starting image to the next pixel, there are a lot of options: we could change either pixel and we could have any number of insertions between them. The insight is that once we have decided where to move both pixels, it is obvious how many insertions we need to do.

The pseudo-code shown below recursively finds the minimal cost to make `pixels[]` smooth, subject to the constraint that the final pixel in the smoothed version must equal `final_value`:

```
int Solve(pixels[], final_value) {
  if (pixels is empty) return 0

  // Try deleting
  best = Solve(pixels[1 to N-1], final_value) + D

  // Try all values for the previous pixel value
  for (all prev_value) {
    prev_cost = Solve(pixels[1 to N-1], prev_value)
    move_cost = |final_value - pixels[N]|
    num_inserts = (|final_value - prev_value| - 1) / M
    insert_cost = num_inserts * I
    best = min(best, prev_cost + move_cost + insert_cost)
  }
  return best
}
```

To answer the original problem, we just take the minimum value from `Solve` over all possible choices of `final_value`.

Unfortunately, this algorithm will be too slow if implemented exactly like this. Within each call to `Solve`, we are making 257 recursive calls, and we might have to go 100 levels deep. That won't

finish in any of our life times, let alone within the time limit! Fortunately, the only reason it is this slow is because we are duplicating work. There are only `256 * N` different sets of parameters that we will ever see for the `Solve` function, so as long as we store the result in a cache, and re-use it when we see the same set of parameters, everything will be much faster. This trick is called Dynamic Programming or more specifically, Memoization.

The Fancy Solution

The run-time of the previous solution is `O(256 * 256 * N)`, which is plenty fast in a competitive language. (Some interpreted languages are orders of magnitude slower at this kind of work than compiled languages - beware!) The extra 256 factor comes from the fact that we need to try 256 possibilities within each function call. It is actually possible to solve this problem in just `O(256 * N)` time. Here are some hints in case you are interested:

- As before, you want to calculate `Cost[n][p]`, the cost of making the first `n` pixels smooth while setting the final pixel to value `p`. Unlike before, you want to do a batch of these updates at the same time. In particular, you want to simultaneously calculate *all* values for `n+1` given the values for `n`.
- So how do we do this batch update? First, let's do an intermediate step to calculate `Cost'[n][p]`, the minimum cost for each value after doing all insertions between pixel `n` and pixel `n+1`. To make this more tractable, it helps to notice that there is never any need to insert a pixel with distance less than `M` from the previous pixel. (Do you see why?)
- The real challenge is that when calculating `Cost[n][]` from `Cost'[n][]`, you are going to want to take minimums over several elements. For example, `Cost[n][q] = min(Cost[n-1][q], {Cost'[n][q-M], Cost'[n][q-M+1], Cost'[n][q-M+2], ..., Cost'[n][q+M]})`. In other words, given the array `Cost'[n][]`, you are going to need to be able to calculate in linear time the minimum element in each range of length `2M+1`. This is an interesting and challenging problem in its own right, and we encourage you to think it through! For a more thorough discussion of this sub-problem, see here.

Code Jam 2010 - Round 1A

Analysis: Number Game

Let us begin by focusing on a single game. Given **A** and **B**, we can first assume without loss of generality that $A \geq B$. Now, how can we decide if it is a winning position or not? (It is impossible to have a tie game since $A+B$ is always decreasing.) Well, a position is winning if and only if, in one step, you can reach a losing position for your opponent. This is an important fact of combinatorial games, so make sure you understand why it's true!

Observations, easy and not so easy

One trivial observation is that (A, A) is a losing position.

Another observation is much trickier unless you have already been exposed to combinatorial games before:

> If $A \geq 2B$, then (A, B) is a winning position.

To justify this: in such a position, suppose k is the largest number of B's we can subtract from A, i.e., $A - kB \geq 0$ and $A - (k+1)B < 0$. We do not know yet whether $(A-kB, B)$ is a winning position or not. But there are just two possibilities. If it is losing, great, we can subtract kB from A, and hand the bad position to our opponent. On the other hand, if it is winning, we can subtract $(k-1)B$ from A, and our opponent has no choice but to subtract another B from the result, giving us the winning position $(A-kB, B)$. Therefore, (A, B) is a winning position either way!

Expand further

The observation above gives us a fairly quick algorithm to figure out who wins a single game (A, B). Instead of using dynamic programming to solve the subproblem for all (A', B') with $A' \leq A$ and $B' \leq B$, which is the most common way of analyzing this kind of game, we can do the following:

- In round 1: If $A \geq 2B$, it's a winning position and we're done. Otherwise, we have only one choice: subtract B from A, and give our opponent $(B, A-B)$.
- In round 2: If $B \geq 2(A-B)$, it is a winning position for our opponent. Otherwise, the only choice he has is to subtract $A-B$ from B, and hand us $(A-B, 2B-A)$.
- In round 3: If $A-B \geq 2(2B-A)$, it is a winning position for us again. Otherwise, we are going to make it $(2B-A, 2A-3B)$.

And so on. This leads to the following algorithm for efficiently solving a single game (A, B), assuming $A \geq B$:

```
bool winning(int A, int B) {
  if (B == 0) return true;
  if (A >= 2*B) return true;
  return !winning(B, A-B);
}
```

Does this sound familiar? One connection you might see is that the Number Game closely resembles Euclid's algorithm for greatest common divisor. It is not hard to see that this algorithm, like Euclid's, will need to recurse at most $O(\log A)$ times.

Unfortunately, we still cannot afford to run the algorithm for every possible (A, B)! To solve the problem, we need to work with many positions at once. Let us go through the same rounds, but imagine having a fixed B and consider all possible A's at the same time:

- Round 1: (A, B). If $A \geq 2B$, i.e., $A/B \geq 2$, then (A, B) is winning.
- Round 2: $(B, A-B)$. If $B \geq 2(A-B)$, i.e., $A/B \leq 3/2$, then (A, B) is losing.
- Round 3: $(A-B, 2B-A)$. If $A-B \geq 2(2B-A)$, i.e., $A/B \geq 5/3$, then (A, B) is wining.
- Round 4: $(2B-A, 2A-3B)$. If $2B-A \geq 2(2A-3B)$, i.e., $A/B \leq 8/5$, then (A, B) is losing.
- And so on.

This gives a fast enough solution to our problem. For each B, we consider all A's in the above manner, and in $O(\log 10^6)$ rounds, we can classify all the A's.

Simplify

The above method is perfectly correct, but it can be made simpler. First of all, does anything in the above list look familiar? There are Fibonacci numbers all over the place! Let $F(i)$ be the i-th Fibonacci number. One can show by induction that the previous analysis is actually saying the following:

- Round 2t-1: If $A/B \geq F(2t+1)/F(2t)$, then (A,B) is a winning position.
- Round 2t: If $A/B \leq F(2t+2)/F(2t+1)$, then (A,B) is a losing position.

It turns out that both $F(2t+1)/F(2t)$ and $F(2t+2)/F(2t+1)$ approach the golden ratio $\varphi = (1 + \sqrt{5}) / 2$ as t gets large. This means there is a very simple characterization of all winning positions!

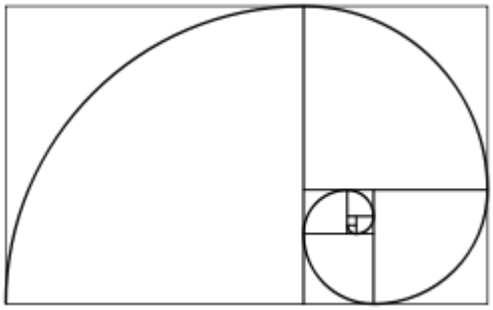
> **Theorem:** (A, B) is a winning position if and only if $A \geq \varphi B$.

Using this theorem, it is easy to solve the problem. Loop through each value for B, and count how many A values satisfy $A \geq \varphi B$.

Why it is Golden?

Once we have stumbled upon the statement of this theorem, it is actually pretty easy to prove. Here is one method: Using mathematical induction, assume we proved the theorem for all smaller A's and B's. If $A \geq 2B$, then (A, B) is a winning position as we discussed earlier. Otherwise we will leave our opponent with $(B, A-B)$. Then (A, B) is winning if and only if $(B, A-B)$ is losing. By our inductive hypothesis, this is equivalent to $B \leq \varphi (A - B)$, or $A \geq ((1 + \varphi) / \varphi) * B$. Since $\varphi = (1 + \varphi) / \varphi$, this proves (A, B) is winning if and only if $A \geq \varphi B$, as required.

Here is another, more geometric viewpoint. You start with a piece of paper which is an A by B rectangle ($A \geq B$), and cut out a B by B square from it. If the rectangle is *golden*, where $A = \varphi B$, then the resulting rectangle will also be golden. In our game, A and B are integers, so the rectangle is never golden. We call it *thin* if $A > \varphi B$, otherwise we call it *fat*. From a thin rectangle you can always cut it to a fat rectangle, and from a fat one you can only cut it to a thin one. They correspond to the winning positions and losing positions, respectively.



A picture of golden rectangles from Wikipedia.

Other Approaches

There are many ways of arriving at the solution to this problem -- our analysis focuses on only one of these ways. Another approach would be to start with a slower method and compute which `(A, B)` are winning positions for small `A`, `B`. From looking at these results, you could easily guess that `(A, B)` is winning if and only if `A ≥ x B` for some `x`, and all that remains would be to figure out what `x` is!

More Information

Euclid's Algorithm - Fibonacci Numbers - Golden Ratio

Code Jam 2010 - Round 1A

Analysis: Rotate

This is a relatively straightforward simulation problem -- the problem statement tells you what to do, and you just need to do it.

Well, except for one fun point: The name of the problem is *Rotate*, and in the statement we talk about the rotation a lot. However, that is the one thing you do *not* need to implement! Rotating 90 degrees clockwise and pushing everything downwards has the same effect as pushing everything towards the right without rotating. As long as you push the pieces in the correct direction, it doesn't matter whether you actually do the rotation. Any **K** pieces in a row will be the same in these two pictures, and your output will be the same too.

So, a simple solution to this problem looks like this:
(1) In each row, push everything to the right. This can be done with some code like the following:

```
for (int row = 1; row < n; ++row) {
  int x = n-1;
  for (int col = n-1; col >= 0; col--)
    if (piece[row][col] != '.') {
      piece[row][x] = piece[row][col]; x--;
    }
  while(x>=0) {piece[row][x]='.'; x--;}
}
```

(2) Test if there are **K** pieces in a row of the same color. There are tricks that can be done to speed this up, but in our problem, **N** is at most 50, and no special optimizations are needed. For each piece, we can just start from that piece and look in all 8 directions (or we can do just 4 directions because of symmetry). For each direction, we go **K** steps from the starting piece, and see if all the pieces encountered are of the same color. The code -- how to go step by step in a direction, and how to check if we are off the board -- will look similar in many different programming languages. We encourage you to check out some of the correct solutions by downloading them from the scoreboard.